

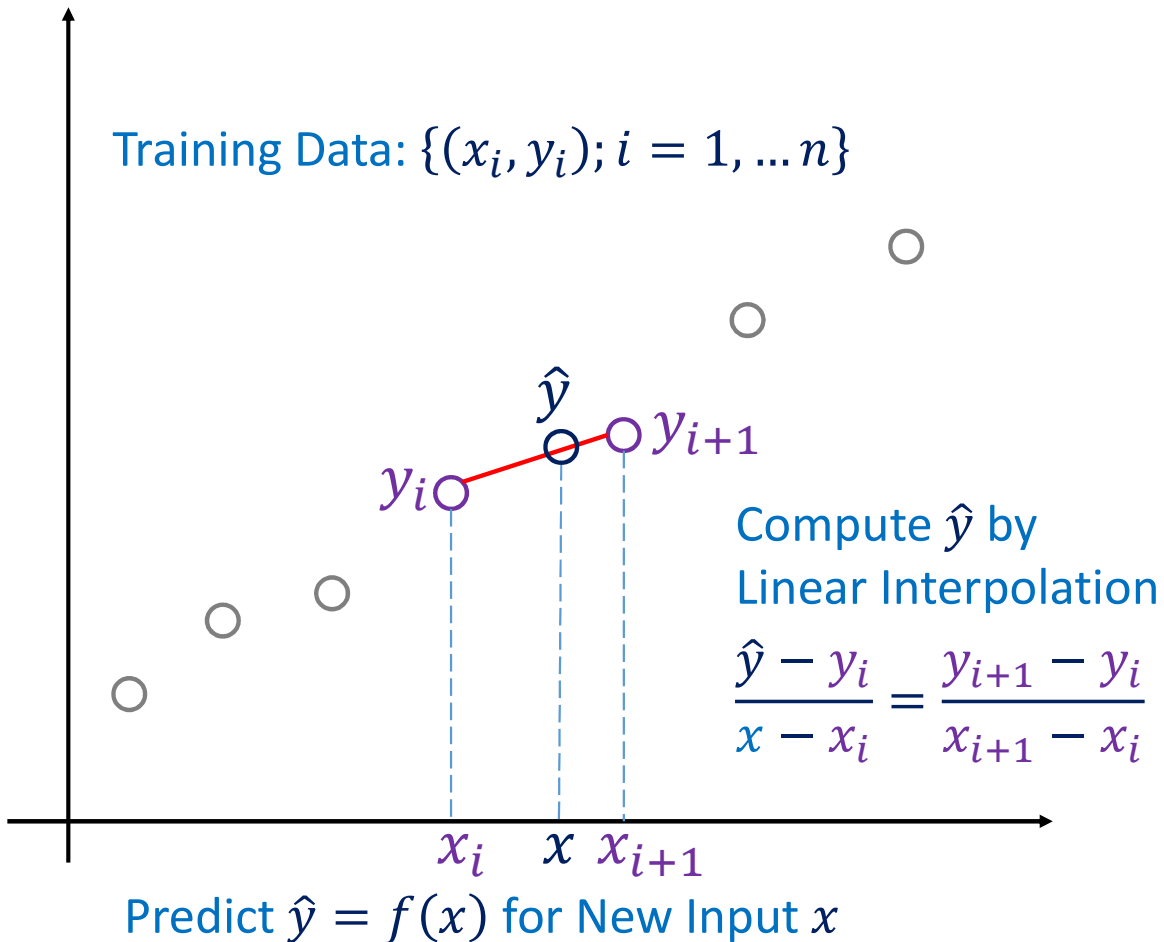
K-Nearest Neighbors



The Lazy Learner

Prithwijit Guha
Dept. of EEE, IIT Guwahati

Regression for Function of Single Variable



No Knowledge about $f(x)$

A Model Free Technique Without Any Learning

Assumption of *Local* Linearity
Neighboring Instances of x are used

Requires *Efficient Module* for *Neighborhood Search*

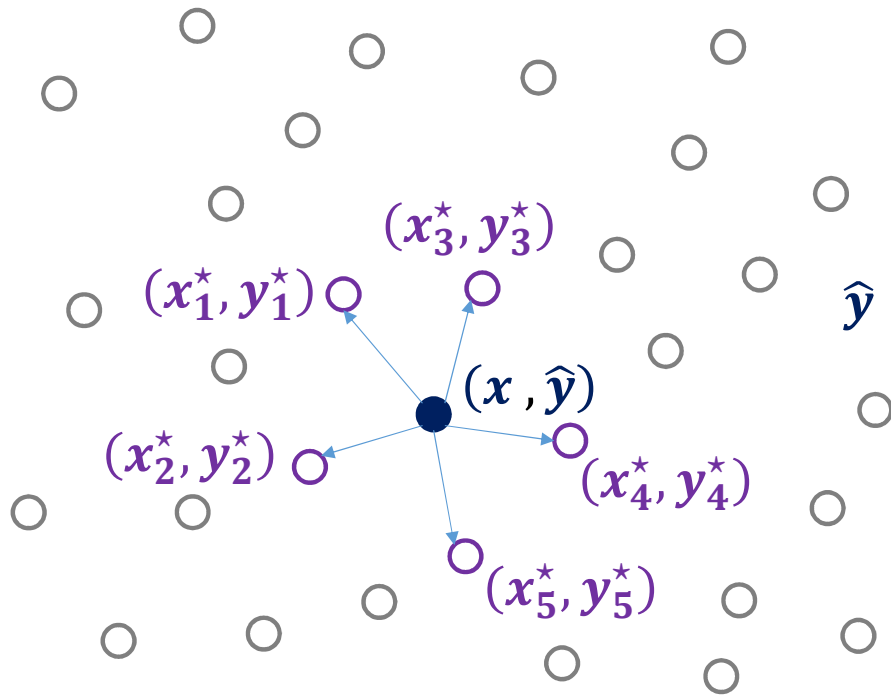
Needs the Train Data for Operation

Good Interpolation but *Poor Extrapolation* Performance

Regression for Vector Function of Multiple Variables

Training Data: $\{(\mathbf{x}_i, \mathbf{y}_i); i = 1, \dots, n\}$

Where $\mathbf{x}_i \in \mathbb{R}^d; \mathbf{y}_i \in \mathbb{R}^m$



Predict $\hat{\mathbf{y}} = \mathbf{F}(\mathbf{x})$ for New Input $\mathbf{x}$

$$\hat{\mathbf{y}} = \frac{1}{k} \sum_{j=1}^k \mathbf{y}_j^*$$

A *Model Free* (No knowledge of $\mathbf{F}(\mathbf{x})$)
Technique *Without Any Learning*

Locate k *Nearest Neighbors* of $\mathbf{x}$ as
 $\{(\mathbf{x}_j^*, \mathbf{y}_j^*); j = 1, \dots, k\}$ ($k = 5$ in the
illustration).

Use some *Distance Measure* and an
Efficient Neighbor Search Module

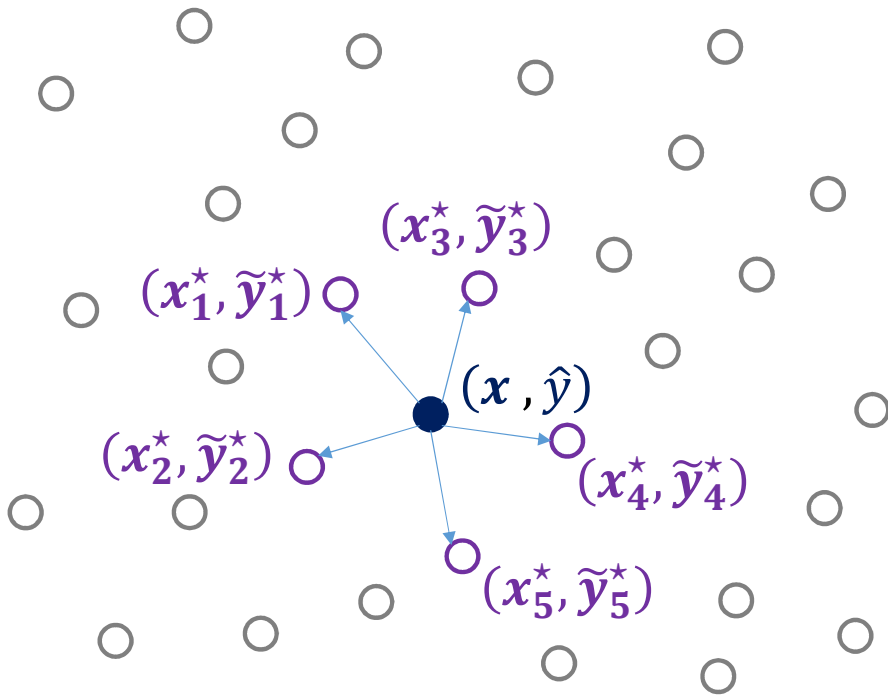
Compute $\hat{\mathbf{y}}$ as Average of
 k Neighboring Values

Needs All Train Data for Operation
with *Good Interpolation* but *Poor*
Extrapolation Performance

Classification using k Nearest Neighbors

Training Data: $\{(\mathbf{x}_i, y_i); i = 1, \dots, n\}$

Where $\mathbf{x}_i \in \mathbb{R}^d; y_i \in \{1, 2, \dots, m\}$



Predict $\hat{\mathbf{y}} = \mathbf{F}(\mathbf{x})$ for New Input $\mathbf{x}$

Construct *One-Hot-Encoded* vectors $\tilde{\mathbf{y}}_i \in \{0, 1\}^m$
from *Label Information* $y_i \in \{1, 2, \dots, m\}$

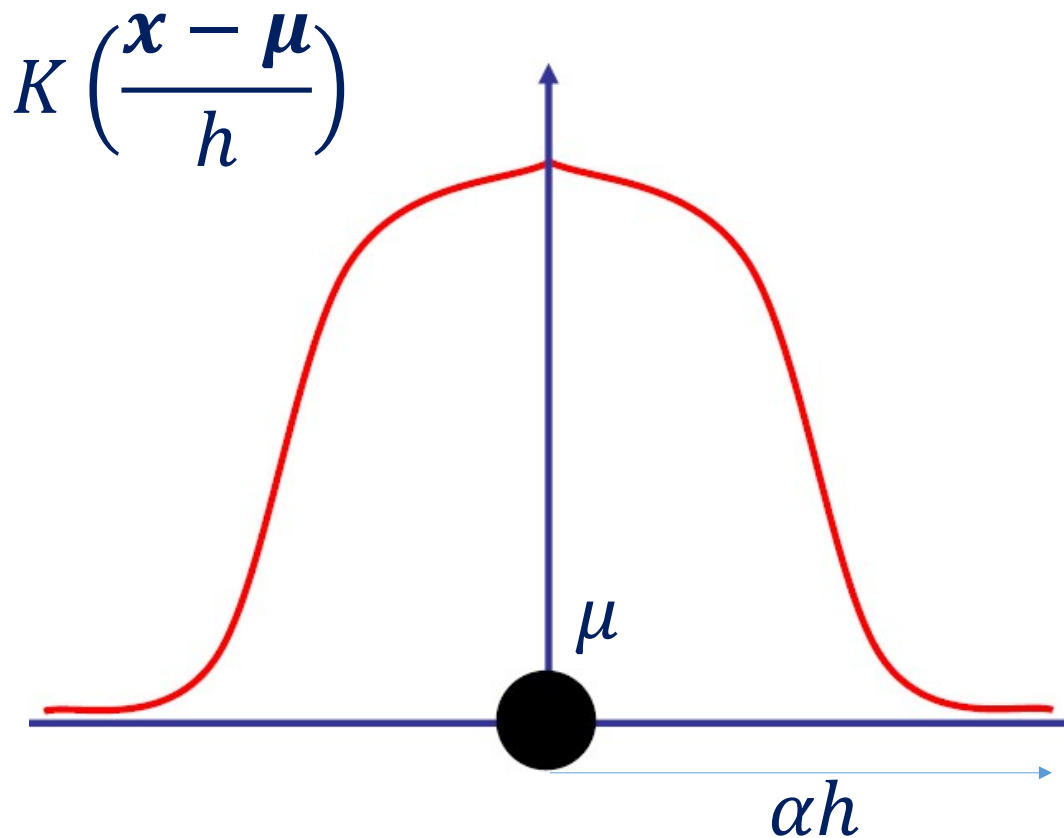
$$\tilde{\mathbf{y}}_i[j] = \begin{cases} 1, & y_i = j \\ 0, & y_i \neq j \end{cases}$$

New Train Data: $\{(\mathbf{x}_i, \tilde{\mathbf{y}}_i); i = 1, \dots, n\}$
Where $\mathbf{x}_i \in \mathbb{R}^d; \tilde{\mathbf{y}}_i \in \{0, 1\}^m$

Predict Class Label $\hat{\mathbf{y}}$ of $\mathbf{x}$ by using the k *Nearest Neighbors*

$$\tilde{\mathbf{y}} = \frac{1}{k} \sum_{j=1}^k \tilde{\mathbf{y}}_j^* \longrightarrow \hat{\mathbf{y}} = \arg \max_{j=1 \dots m} \tilde{\mathbf{y}}[j]$$

Kernel Functions



Decays to *ZERO* As Moves
Away From μ

Rate of Decay Controlled by
Bandwidth h

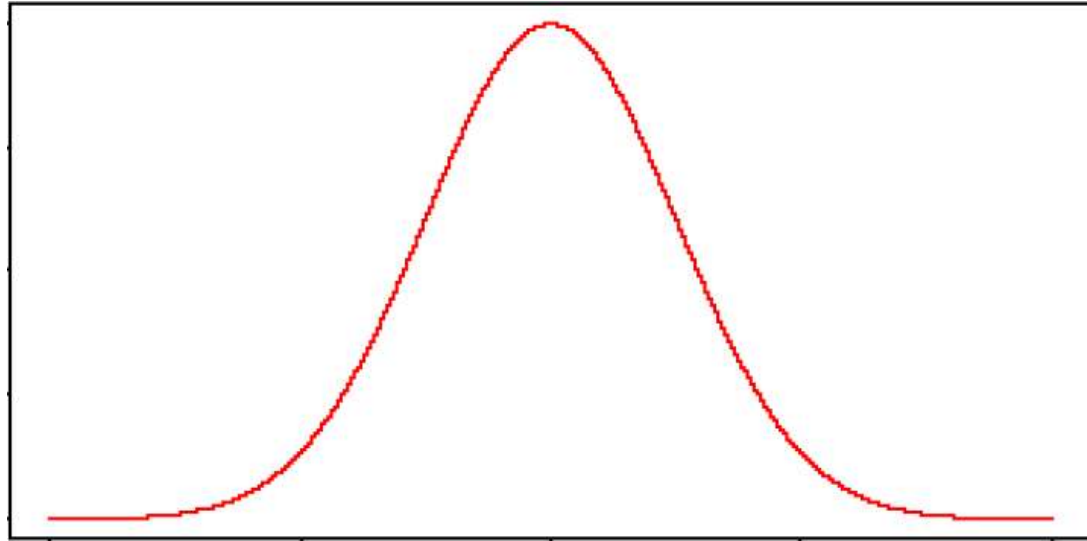
Maximum at μ

Symmetric around μ

$$\int_{\mathbb{R}^d} K\left(\frac{x - \mu}{h}\right) dx = 1$$

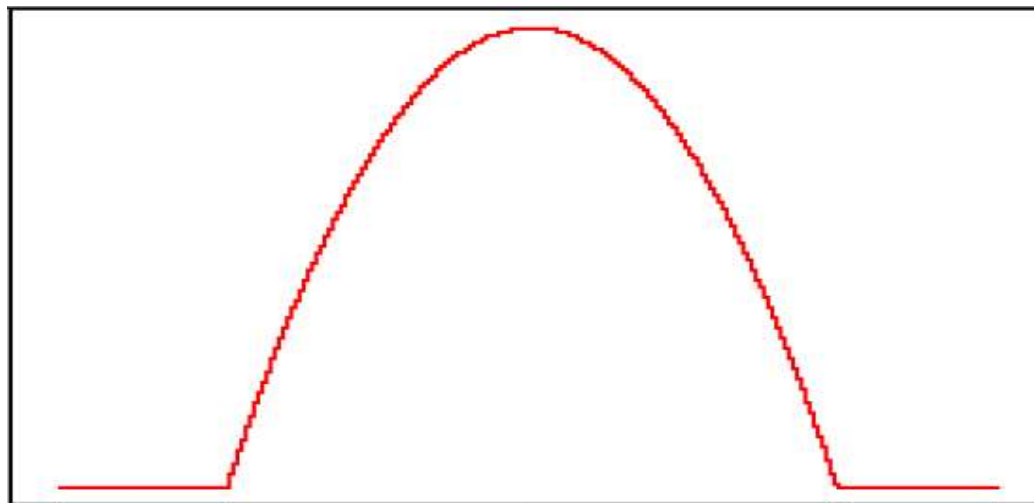
Gaussian Kernel Function

$$K_N\left(\frac{\mathbf{x} - \boldsymbol{\mu}}{h}\right) = c_N \exp\left(-\frac{1}{2} \left\|\frac{\mathbf{x} - \boldsymbol{\mu}}{h}\right\|^2\right)$$

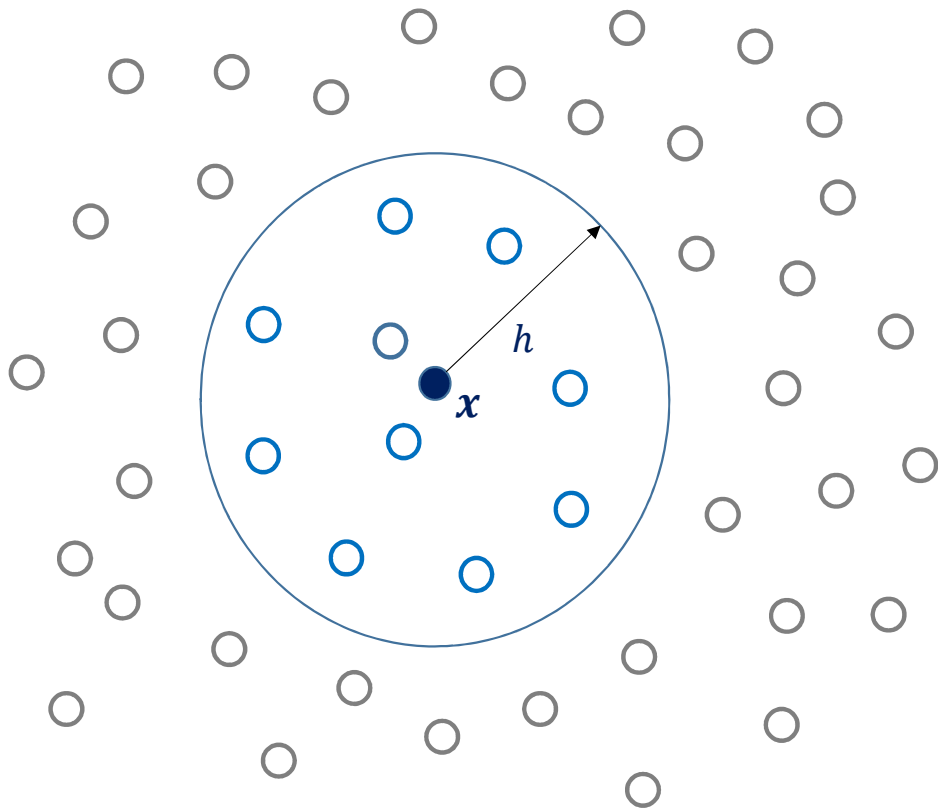


Epanechnikov Kernel Function

$$K_E\left(\frac{\mathbf{x} - \boldsymbol{\mu}}{h}\right) = \begin{cases} c_E \left(1 - \left\|\frac{\mathbf{x} - \boldsymbol{\mu}}{h}\right\|^2\right), & \|\mathbf{x} - \boldsymbol{\mu}\|_2 \leq h \\ 0, & \|\mathbf{x} - \boldsymbol{\mu}\|_2 > h \end{cases}$$



Regression with Kernel Weighted Neighbors



Training Data: $\{(\mathbf{x}_i, \mathbf{y}_i); i = 1, \dots, n\}$
Where $\mathbf{x}_i \in \mathbb{R}^d; \mathbf{y}_i \in \mathbb{R}^m$

Predict $\hat{\mathbf{y}} = F(\mathbf{x})$ for New Input $\mathbf{x}$

$$\hat{\mathbf{y}}(\mathbf{x}) = \frac{\sum_{i=1}^n \mathbf{y}_i K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)}{\sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)}$$

Points $\mathbf{x}_i$ further away from Input $\mathbf{x}$ has lower weight assigned by Kernel $K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$.
The nearer points get higher weight.

Classification with Kernel Weighted Neighbors

Training Data: $\{(\mathbf{x}_i, y_i); i = 1, \dots, n\}$

Where $\mathbf{x}_i \in \mathbb{R}^d; y_i \in \{1, 2, \dots, m\}$

Construct *One-Hot-Encoded* vectors $\tilde{\mathbf{y}}_i \in \{0, 1\}^m$
from *Label Information* $y_i \in \{1, 2, \dots, m\}$

$$\tilde{\mathbf{y}}_i[j] = \begin{cases} 1, & y_i = j \\ 0, & y_i \neq j \end{cases}$$

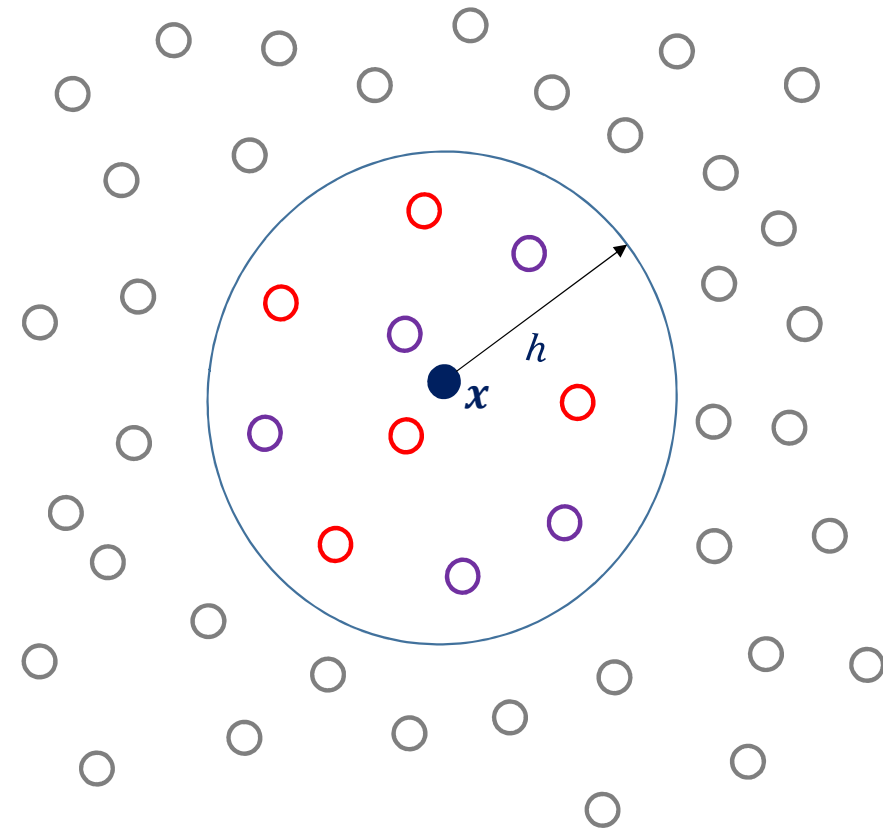
New Train Data: $\{(\mathbf{x}_i, \tilde{\mathbf{y}}_i); i = 1, \dots, n\}$

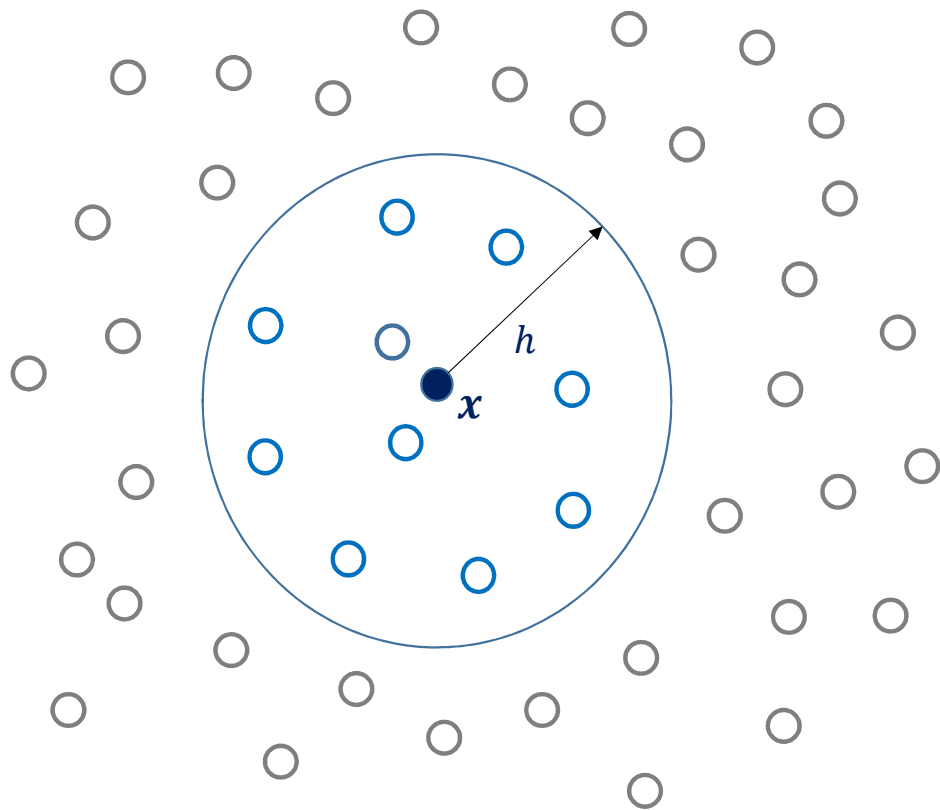
Here, $\mathbf{x}_i \in \mathbb{R}^d; \tilde{\mathbf{y}}_i \in \{0, 1\}^m$

Predict $\tilde{\mathbf{y}}$ using Kernel Weighted Average of
Neighboring $\tilde{\mathbf{y}}_i$

$$\tilde{\mathbf{y}} = \frac{\sum_{i=1}^n \tilde{\mathbf{y}}_i K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)}{\sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)}$$

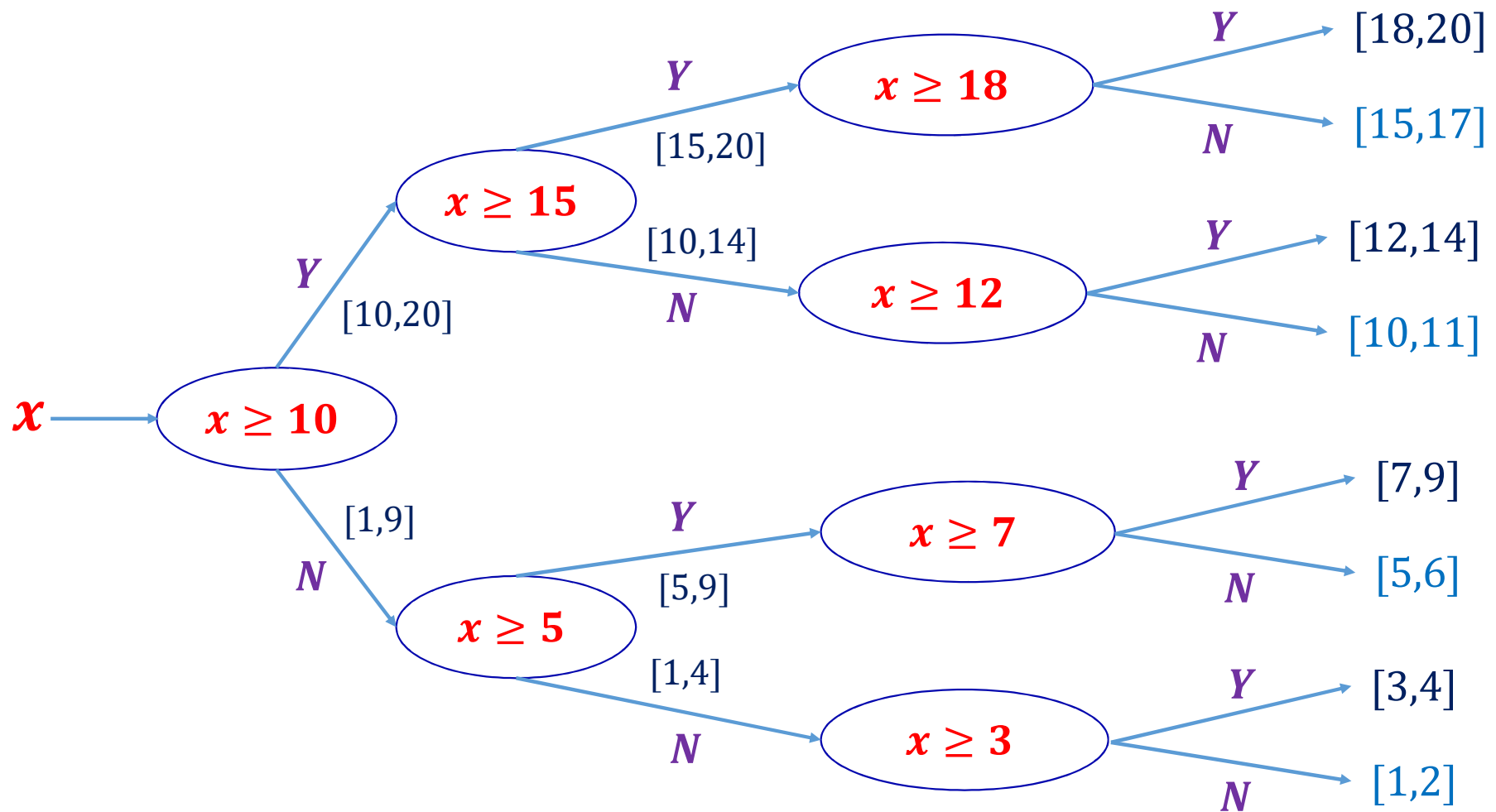
$$\hat{y} = \arg \max_{j=1 \dots m} \tilde{\mathbf{y}}[j]$$





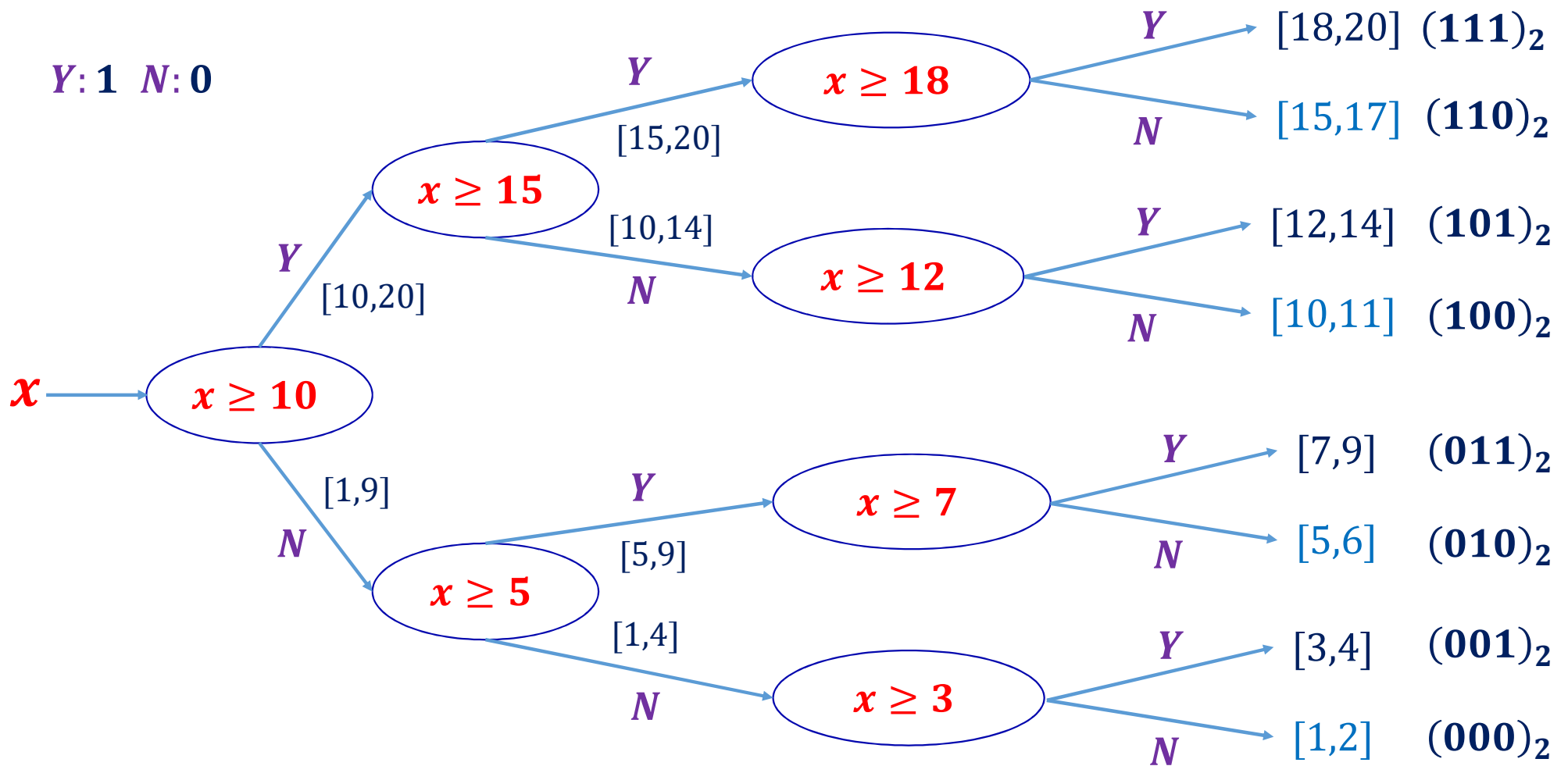
K-D Tree: Searching
for the Neighbors

Guessing a Number in $[1,20]$

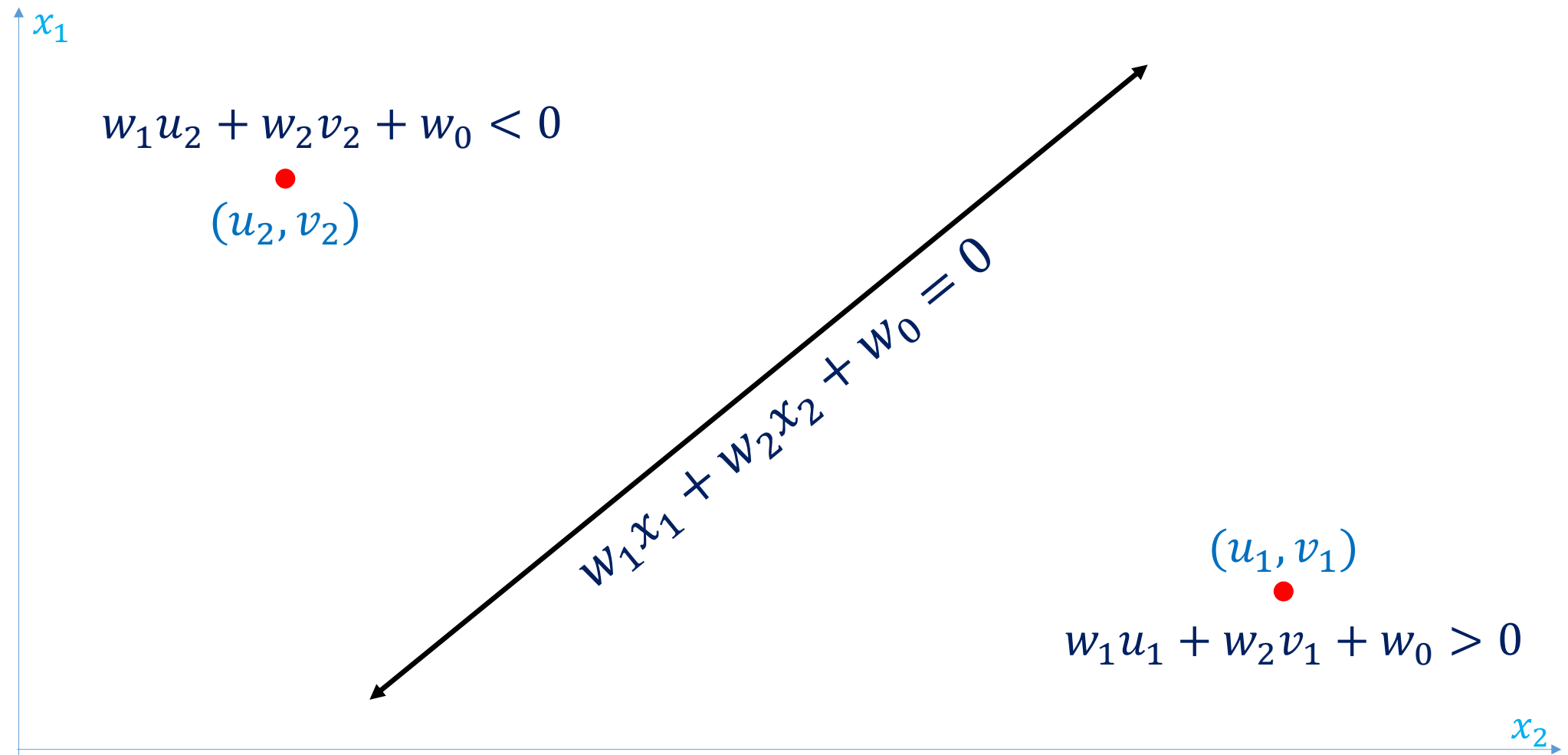


Guessing a Number in $[1,20]$

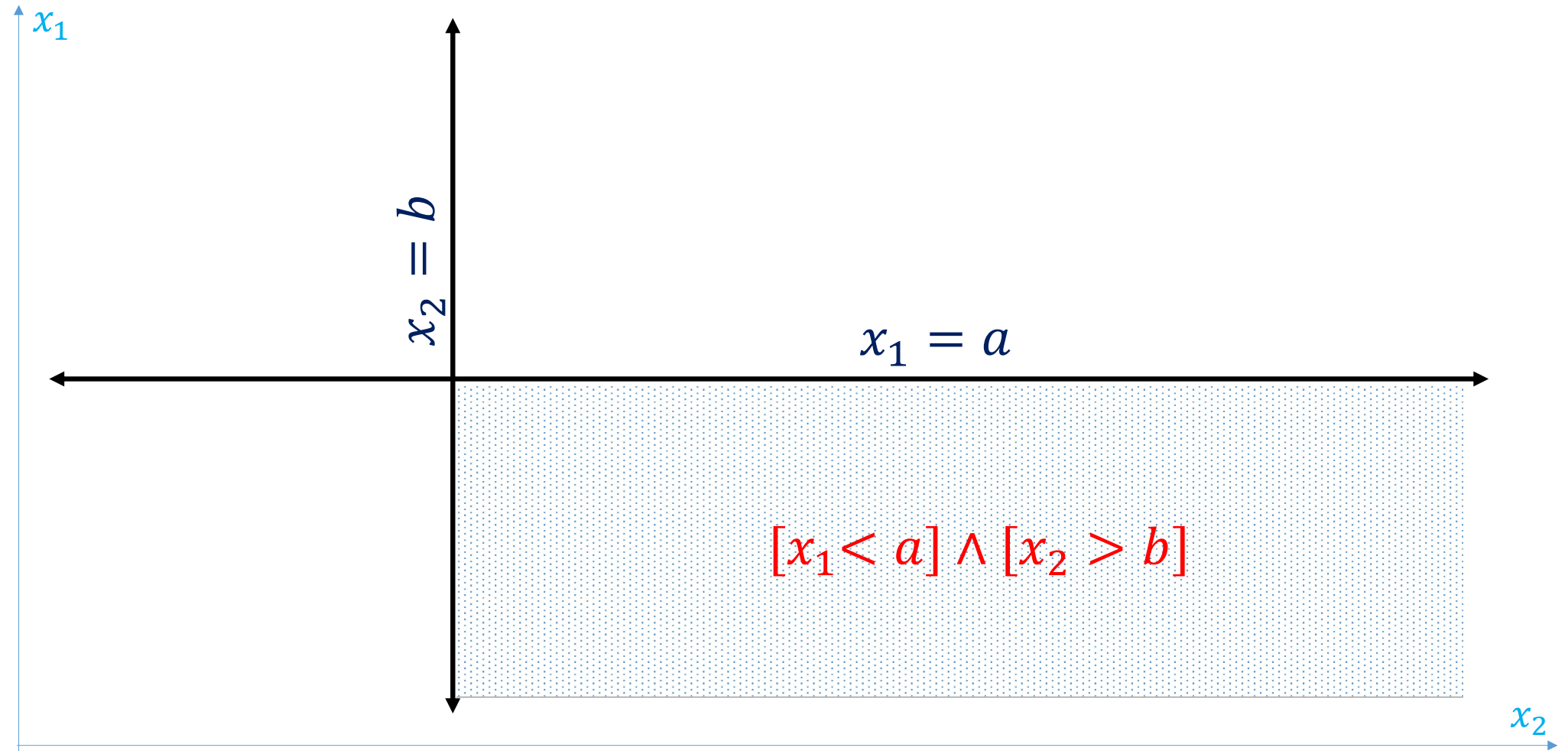
Y: 1 N: 0



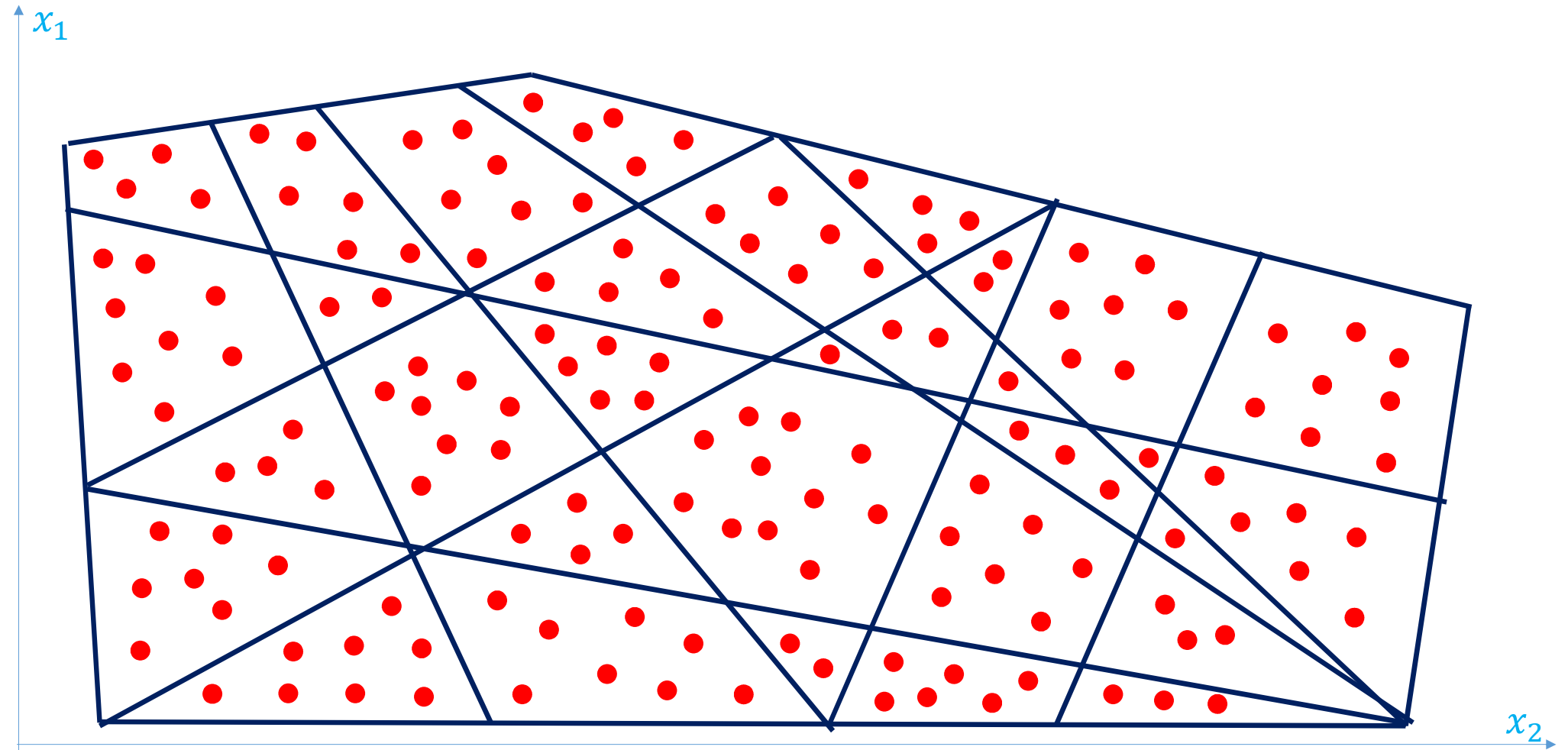
Feature Space Partitioning: Oblique



Feature Space Partitioning: Axis Aligned

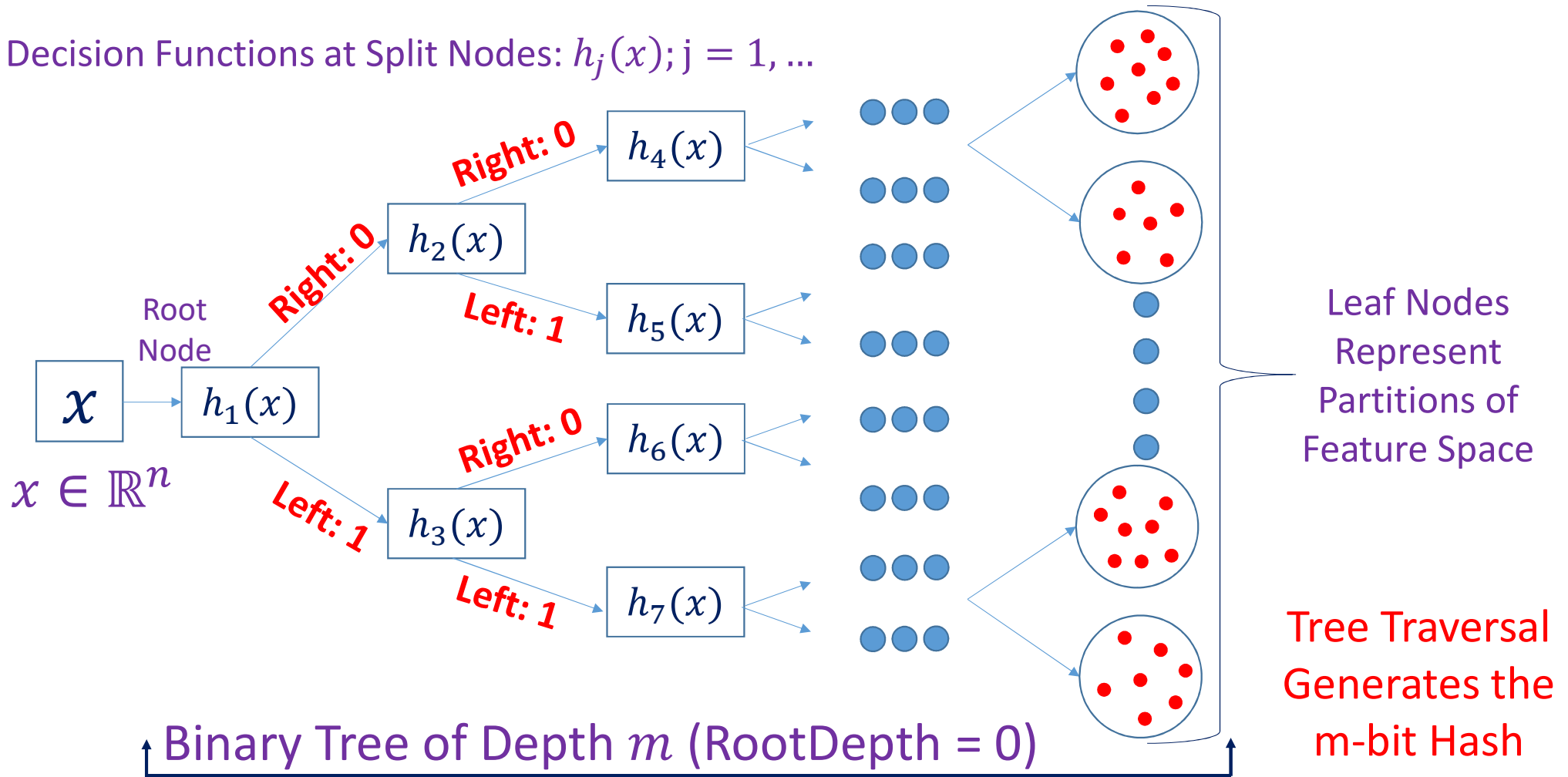


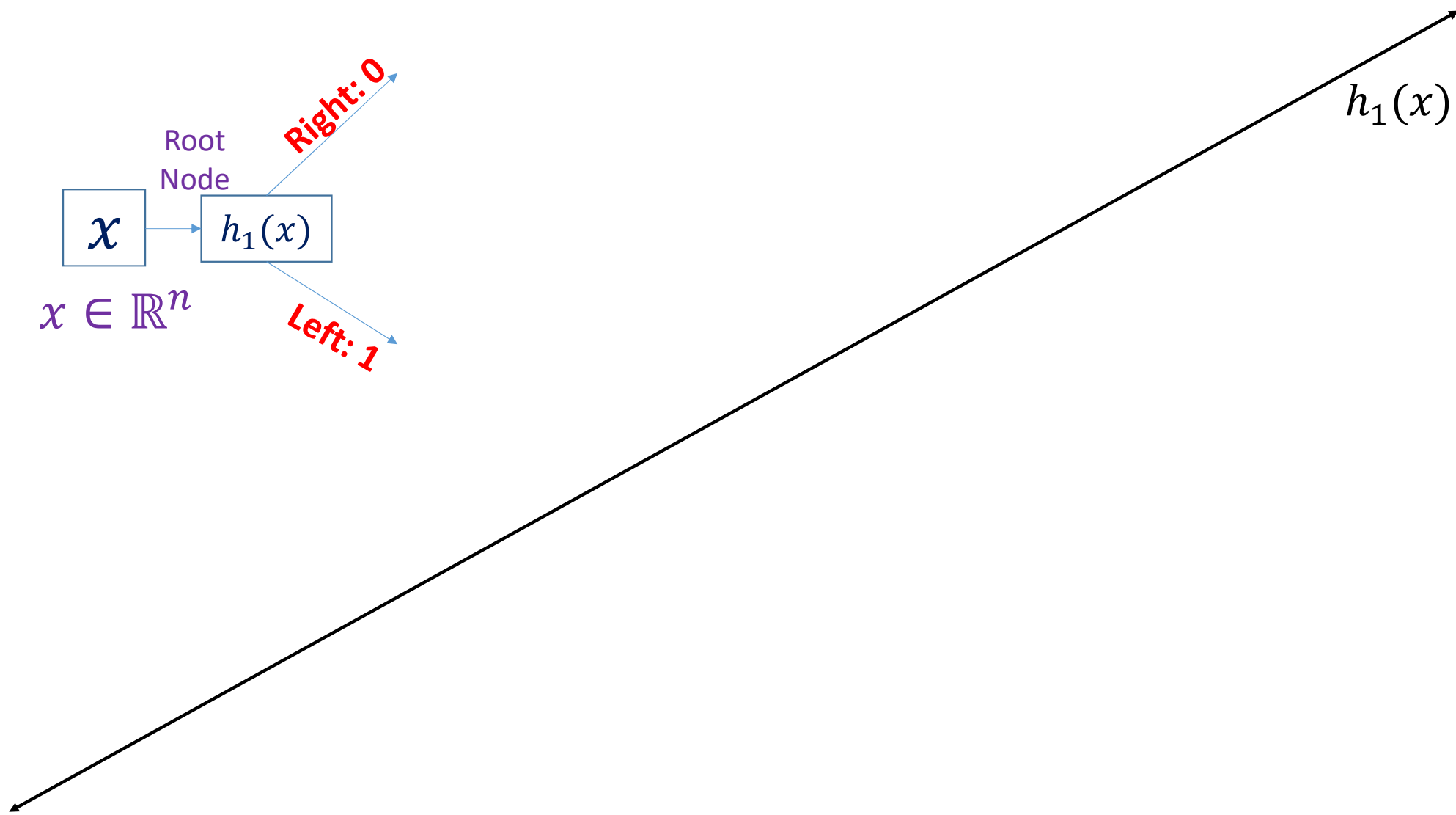
Feature Space Partitioning

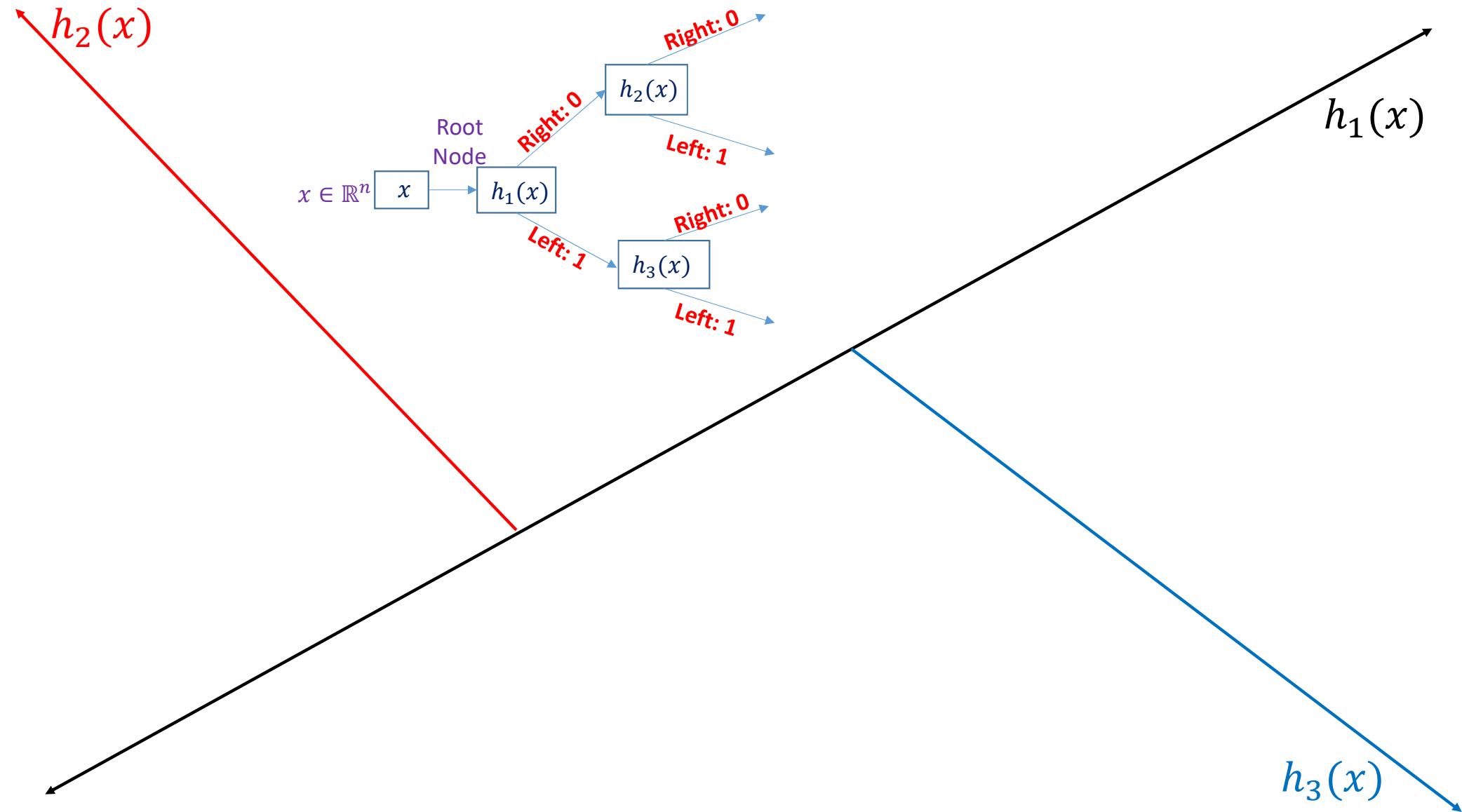


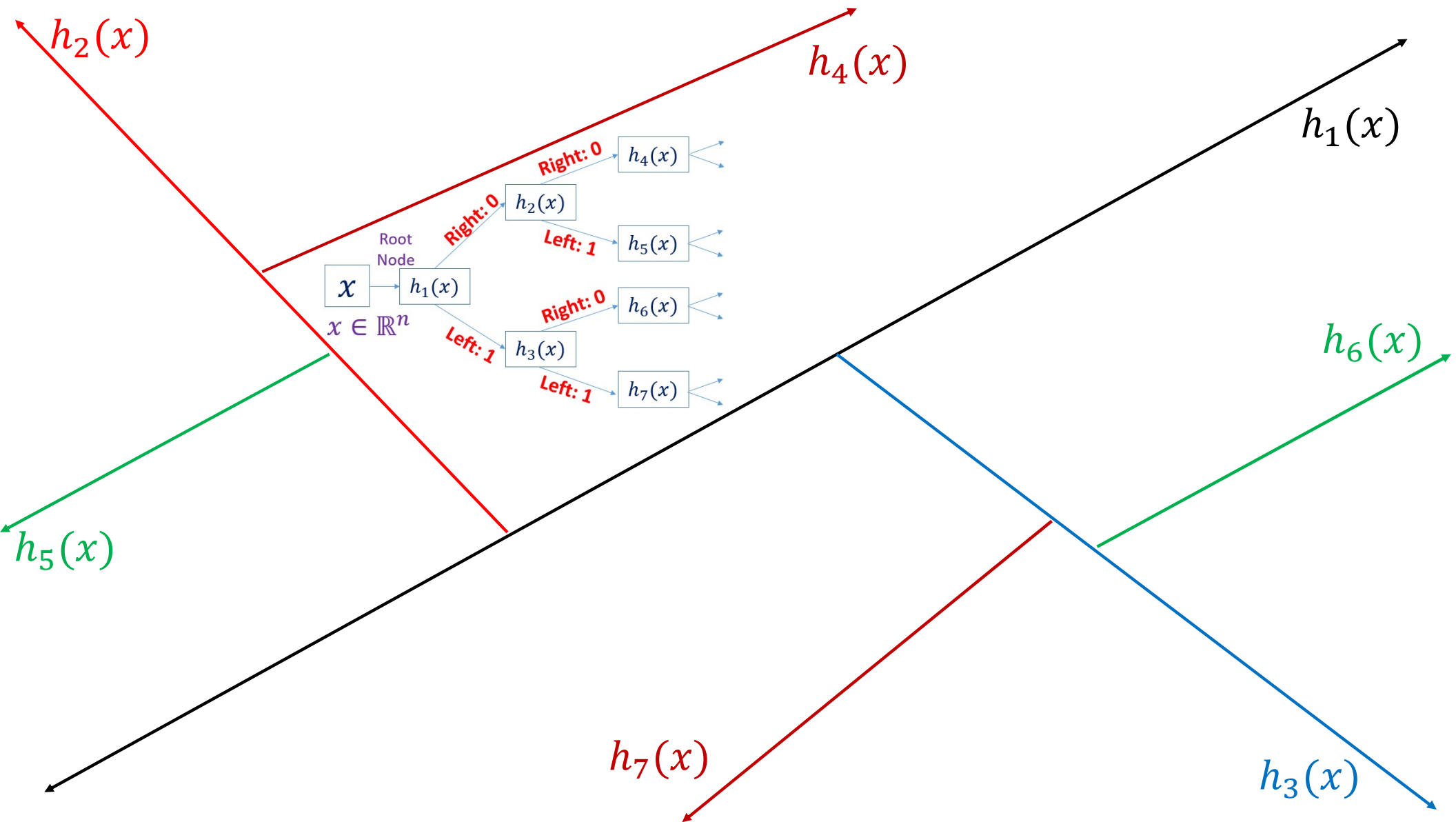
Hashing with Hierarchical Structures

Decision Functions at Split Nodes: $h_j(x); j = 1, \dots$







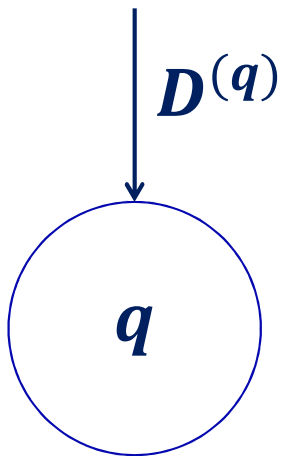


KD-Tree Algorithm: Termination Criterion

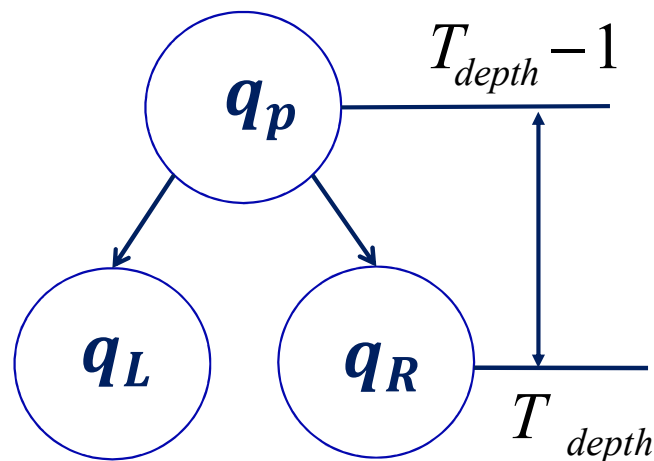
Maximum Tree Depth is T_{depth}^{max}

Input to Node q of Tree T is $D^{(q)} = \{x_i; i = 1, \dots, n_q; x \in \mathbb{R}^d\}$

Minimum Number of Data Instance Necessary for a Split is m



Insufficient Input to
Node $|D^{(q)}| < m$



Nodes at Maximum
Allowable Tree Depth

```
FUNCTION bool  $snc = splitNode(q)$   
IF  $[D^{(q)} < m] \vee [Depth(q) = T_{depth}^{max}]$   
    RETURN FALSE  
ELSE  
    RETURN TRUE  
ENDIF
```

KD-Tree Algorithm

INPUT: $\mathbf{D}^{(q)} = \{\mathbf{x}_i; i = 1, \dots, n_q, \mathbf{x} \in \mathbb{R}^d\}$

Evaluate Split Criteria: $snc = \text{splitNode}(\mathbf{q})$

IF $snc = \text{TRUE}$

Get Dimension of Highest Variance: $j_q = \underset{r=1 \dots d}{\operatorname{argmax}} \operatorname{Var} \{ \mathbf{x}_1[r], \dots, \mathbf{x}_{n_q}[r] \}$

Compute Median: $\theta_q = \operatorname{Median} \{ \mathbf{x}_1[j_q], \dots, \mathbf{x}_{n_q}[j_q] \}$

Construct Decision Function: $h_q(\mathbf{x}): \mathbf{x}[j_q] \geq \theta_q$

IF $\mathbf{x}[j_q] \geq \theta_q$ APPEND $\mathbf{x} \rightarrow \mathbf{D}_R^{(q)}$; ELSE APPEND $\mathbf{x} \rightarrow \mathbf{D}_L^{(q)}$

ENDIF

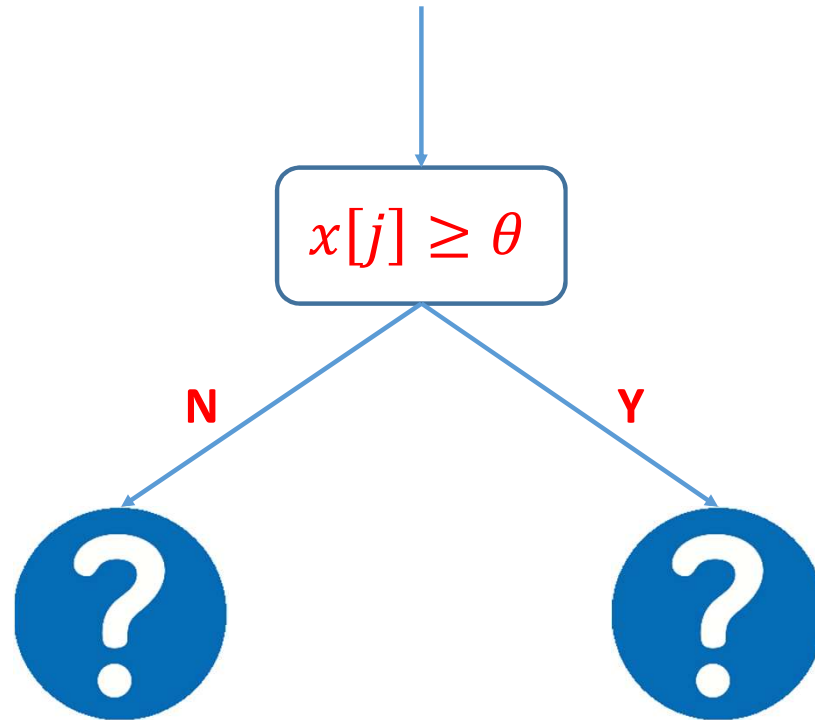
INPUT: $\mathbf{D}_L^{(q)} \rightarrow \mathbf{q}_L$ AND $\mathbf{D}_R^{(q)} \rightarrow \mathbf{q}_R$

The Dataset

6	-4	4	7	0	-7	1	-8	0	-7
8	-9	-4	-5	0	8	10	-1	6	7

KD-Tree: Data Splitting at Root Node

6	-4	4	7	0	-7	1	-8	0	-7
8	-9	-4	-5	0	8	10	-1	6	7



The Dataset: Finding Split Parameters

6	-4	4	7	0	-7	1	-8	0	-7
8	-9	-4	-5	0	8	10	-1	6	7

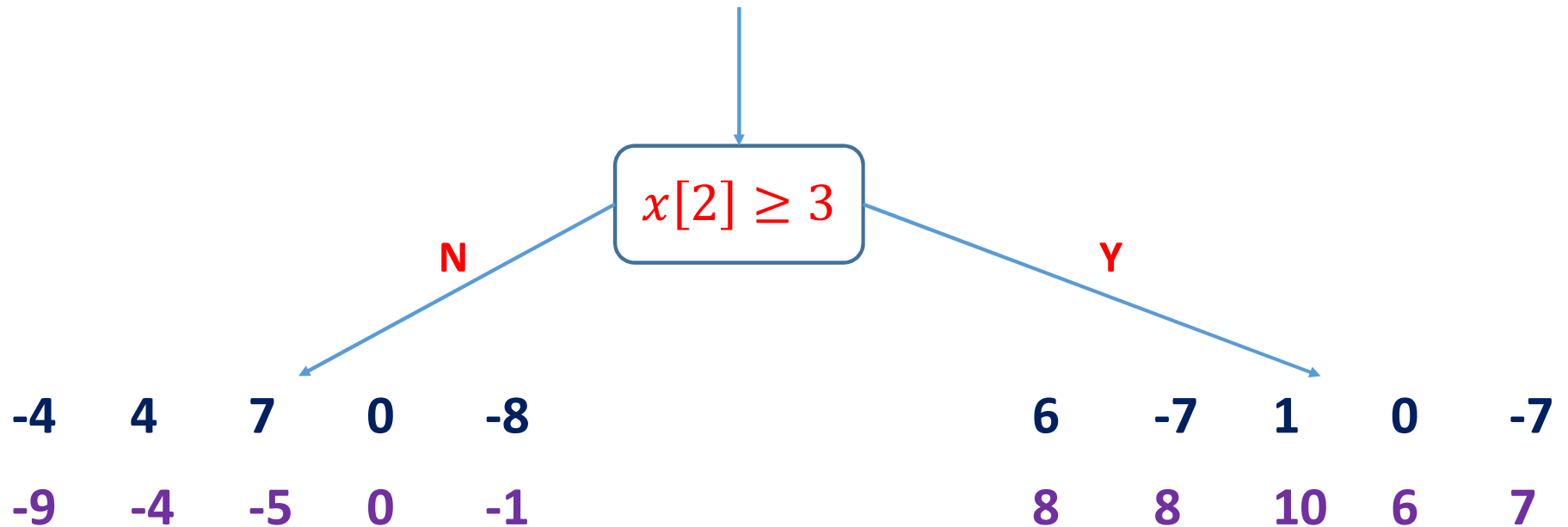
$$d = 1 \quad \sigma_1 = 5.5136$$

$$d = 2 \quad \sigma_2 = 6.6332$$

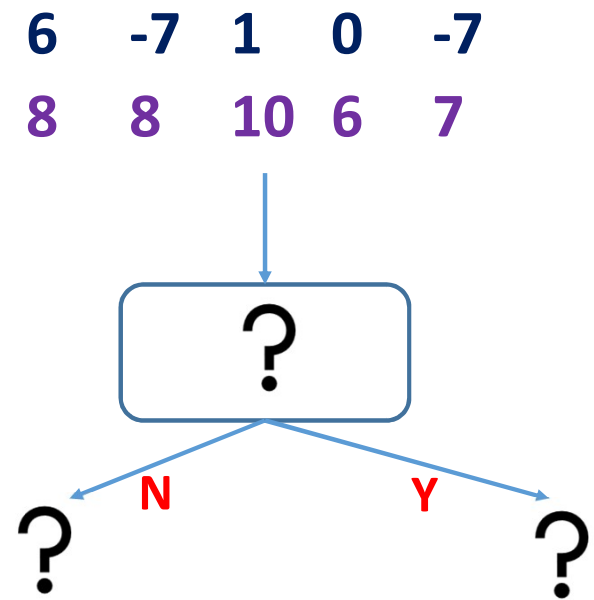
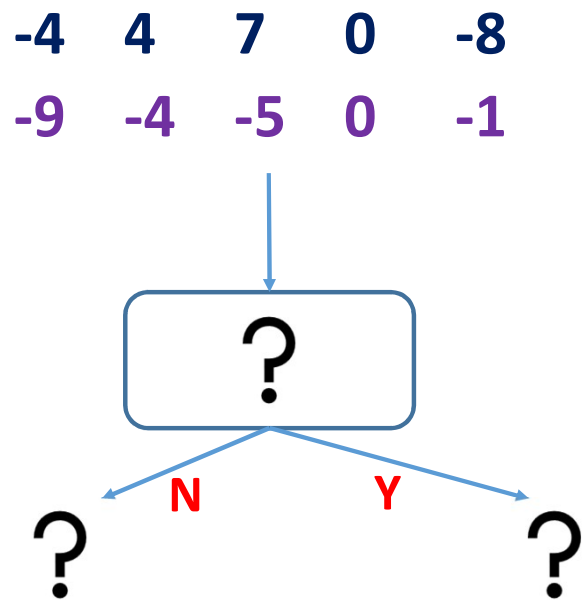
$$\text{median}[d = 2] = 3$$

KD-Tree: Data Splitting at Root Node

6	-4	4	7	0	-7	1	-8	0	-7
8	-9	-4	-5	0	8	10	-1	6	7



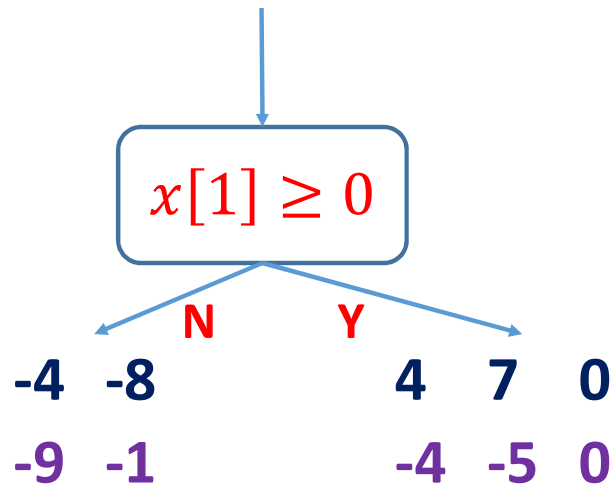
KD-Tree: Further Splitting



KD-Tree: Further Splitting

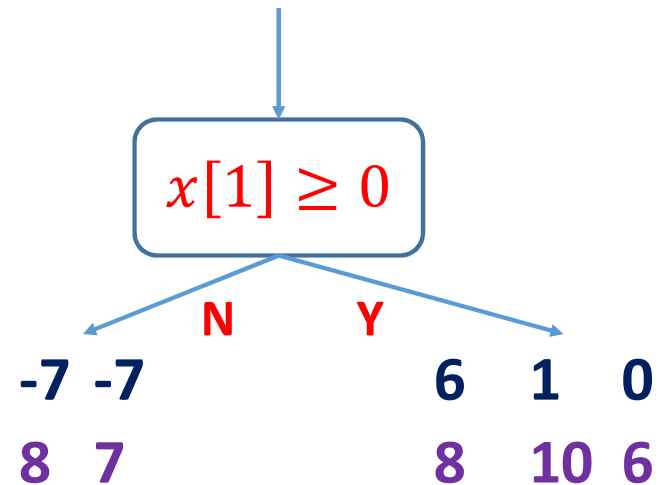
-4 4 7 0 -8 $\sigma_1 = 6.0166$
 -9 -4 -5 0 -1 $\sigma_2 = 3.5637$

$median[d = 1] = 0$

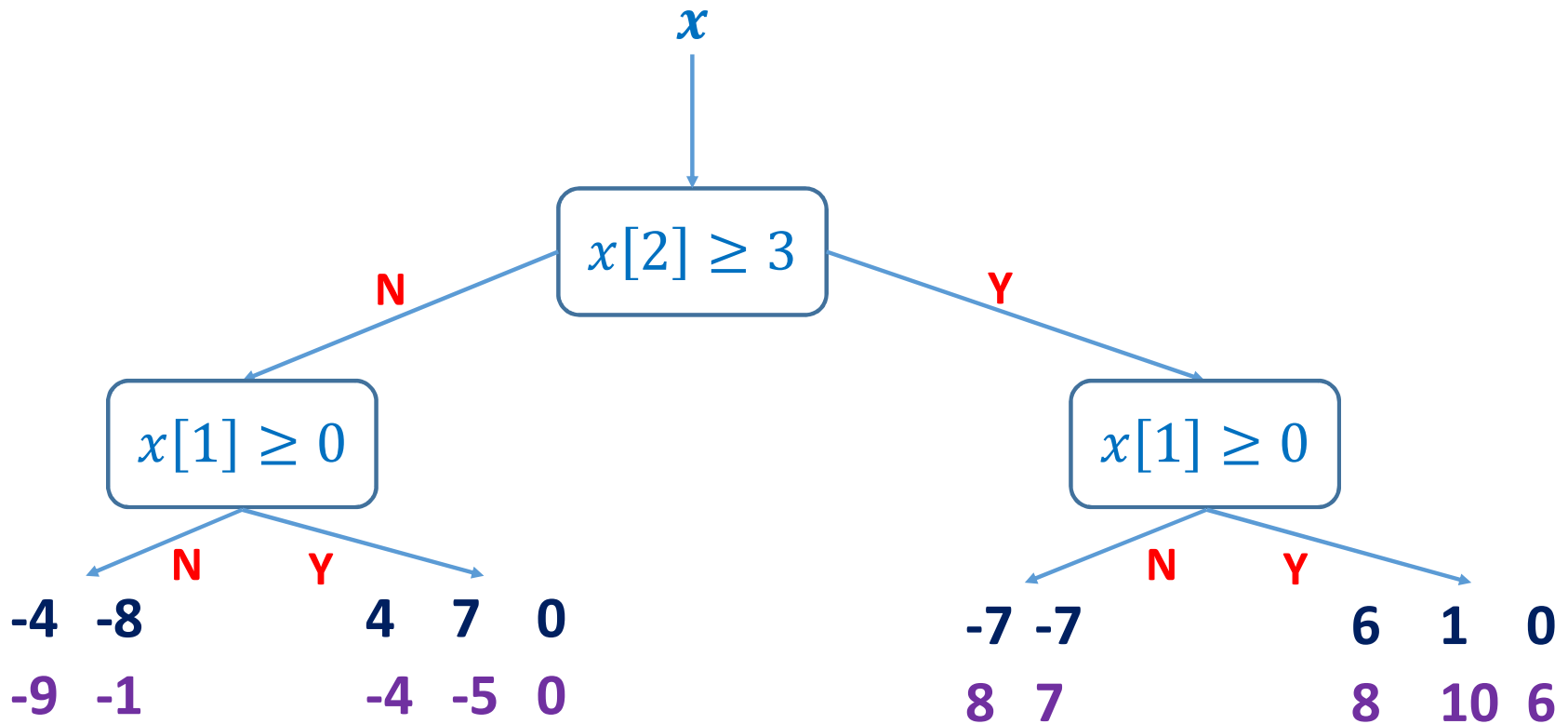


6 -7 1 0 -7 $\sigma_1 = 5.5946$
 8 8 10 6 7 $\sigma_2 = 1.4832$

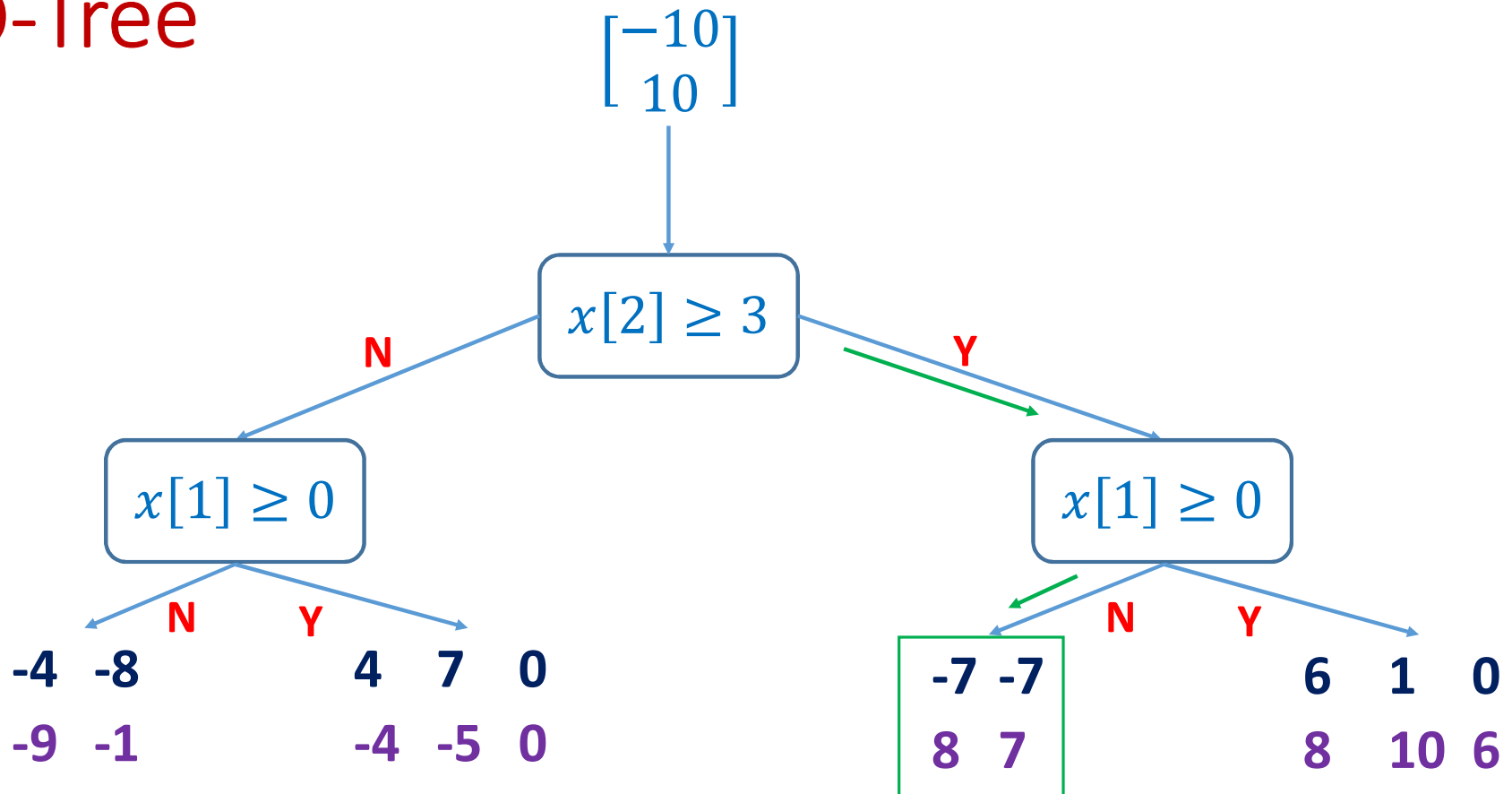
$median[d = 1] = 0$



The Learned K-D Tree



KD-Tree



Summary

- Model-Free Regression and Classification
- Non-parametric Approach
- Lazy Learner
- K-Nearest Neighbors (K-NN)
- K-D Tree
- Position Membership Weighing



Thank You